Adventure Works Auction System - SQL Implementation Report

Group D:

- Ana Martins - 20242039
- António Crespo - 20242034
- João Manso - 20242194
- Pedro Matos – 20242053

Summary

This report documents the design and implementation of a database extension for Adventure Works Bicycles to support an online auction system for stock clearance, plus an analysis to identify optimal locations for new physical stores.

The project consisted of two distinct parts:

1. **Part I**: Extension of the existing Adventure Works database schema to support an online auction system with necessary tables and stored procedures for managing auctions, bids, and winners.
2. **Part II**: Data analysis to identify the two most promising cities in the United States for opening brick and mortar stores without competing with existing high-value retail customers.

Both solutions were implemented using T-SQL for Microsoft SQL Server, following best practices for performance, error handling, and data integrity.

Part I: Auction System Implementation

Database Design Approach

The auction system extension was designed to integrate with the existing Adventure Works database while maintaining separation through a dedicated schema. Our implementation added four primary tables and five stored procedures, all contained within a new Auction schema to maintain clean separation from the core database objects.

Schema Design Considerations

We designed the database extension with the following considerations:

1. **Idempotent Deployment**: Scripts can be run multiple times without errors or duplicate data

2. **Error Handling**: Robust validation and error handling for all operations
3. **Performance**: Added indexes to optimize query performance under high load (especially during Black Friday)
4. **Configuration Management**: Centralized configuration values for easy adjustment
5. **Business Rule Enforcement**: Implementation of all specified business rules for pricing and bidding

Database Schema Implementation

The Auction schema consists of four main tables:

AuctionTable

The primary table containing all active and historical auctions:

- **AuctionID**: Primary key, auto-incrementing identifier
- **ProductID**: Foreign key linking to the product being auctioned
- **InitialBidPrice**: Starting bid price calculated based on product type:
    - 25% discount (75% of list price) for resale products (MakeFlag=0)
    - 50% discount (50% of list price) for in-house manufactured products (MakeFlag=1)
- **CurrentBid**: The current highest bid (if any)
- **Active:** BIT flag indicating if auction is active (1) or inactive (0)
- **Cancelled:** BIT flag for auctions removed before completion (1 = cancelled, 0 = not cancelled)
- **StartTime**: When the auction began
- **ExpireDate**: When the auction automatically ends

BidTable

Records all bids placed for each auction:

- **BidID**: Primary key, auto-incrementing identifier
- **AuctionID**: Foreign key linking to the relevant auction
- **CustomerID**: Foreign key linking to the customer who placed the bid
- **BidAmount**: The monetary amount of the bid

- **BidDate**: Timestamp when the bid was placed

WinnerTable

Records the winners of completed auctions:

- **AuctionID**: Part of composite key linking to the auction
- **BidID**: Part of composite key linking to the winning bid
- **WinnerID**: The customer who won the auction
- **FinalBid**: The final winning bid amount
- **ConclusionDate**: When the auction was concluded

ConfigTable

Stores system-wide configuration values:

- **DefaultMinimumBidIncrement**: Minimum bid increment (default 0.05)
- **DefaultStartBidDate:** Default start date for new auctions (set to November 17, 2025, beginning the two-week campaign period)
- **DefaultEndBidDate:** Default end date for new auctions (set to November 29, 2025, covering the Black Friday period)
- **InitialDiscountResaleProducts:** Default discount for resale products (25%, meaning customer pays 75% of list price)
- **InitialDiscountOwnedProducts:** Default discount for in-house products (50%, meaning customer pays 50% of list price)
- **MaxPriceFactor**: Maximum allowable price as percentage of list price (100%)

These tables are relationally connected as shown in the schema diagram provided, with appropriate foreign key constraints to ensure data integrity.

<u>Stored Procedures</u>

Five stored procedures were implemented to support all required auction operations:

1. uspAddProductToAuction

Adds a product to the auction system with validation checks:

- Verifies product exists and is active (not discontinued)

- Checks product is not already in an active auction
- Ensures product has a valid list price
- Calculates appropriate initial bid price based on product type and specified discount rules:
    - For resale products (MakeFlag=0): InitialBidPrice = ListPrice × 0.75 (applying 25% discount)
    - For in-house products (MakeFlag=1): InitialBidPrice = ListPrice × 0.50 (applying 50% discount)
    - This implementation aligns with the business requirement to offer moderate discounts on resale products and deeper discounts on in-house manufactured products
- Sets appropriate expiration date (default 7 days)

2. uspTryBidProduct

Processes bid attempts with extensive validation:

- Verifies auction exists and is active
- Validates customer ID
- Ensures bid amount exceeds current bid plus minimum increment (default 0.05€)
- Checks that bid does not exceed maximum allowed price (100% of list price by default)
- Records bid in BidTable
- Implements special win condition: automatically declares a winner if the bid reaches or exceeds the maximum price minus the minimum increment (e.g., if maximum is 100€ and increment is 0.05€, a bid of 99.95€ or higher wins)
- Automatically updates auction status to inactive when a winner is declared

3. uspRemoveProductFromAuction

Removes a product from auction:

- Sets the auction as inactive and cancelled
- Preserves bid history for reporting

4. uspListBidsOffersHistory

Returns customer bid history within a date range:

- Returns detailed bid history for a specific customer
- Filters by date range
- Option to show only active auctions or all auction history

5. uspUpdateProductAuctionStatus

Updates auction status and assigns winners:

- Marks expired auctions as inactive
- Identifies the highest bid for each expired auction
- Records winners in the WinnerTable

Implementation Decisions Beyond Requirements

Several implementation decisions were made to enhance the auction system beyond the basic requirements:

1. **Index Creation**: Created indexes on frequently queried columns to improve performance under high load conditions (particularly important for Black Friday).
2. **Auction Cancellation Flag**: Added a dedicated Cancelled flag to distinguish between auctions that ended naturally and those that were administratively removed.
3. **Robust Error Handling**: Implemented comprehensive error handling with specific error messages to assist users:

```
-- Example from uspTryBidProduct
IF @BidAmount < @CurrentBid + @MinimumIncrement
BEGIN
    DECLARE @ErrorMsg NVARCHAR(255);
    SET @ErrorMsg = 'Please insert a greater value than: '
                    + CAST(@CurrentBid + @MinimumIncrement AS
NVARCHAR(50)) + '€';
    THROW 50001, @ErrorMsg, 1;
END
```

4. **Automatic Winner Determination**: Implemented logic to automatically close auctions and declare winners when either the exact maximum price is reached or when a bid is within the minimum increment threshold of the maximum (i.e., MaxPrice - MinimumIncrement). This approach prevents unnecessary bidding once a price very close to the maximum is reached, improving user experience.
5. **Date Boundary Handling**: Ensured auctions couldn't expire after the campaign end date (November 29, 2025), which corresponds to the Black Friday weekend

These design decisions and implementation details ensure that the auction system fulfills its primary business purpose: supporting the two-week stock clearance campaign ending on Black Friday weekend in November 2025. The system incorporates all specified business rules while adding performance optimizations and user experience enhancements that will help Adventure Works efficiently clear inventory before introducing new bicycle models.

Part II: Brick and Mortar Store Location Analysis

Analysis Approach

To identify optimal cities for new Adventure Works brick and mortar stores in the United States, we implemented a data-driven approach following these steps:

1. **Top30Customers**: We identified the 30 customers who spent the most (highest TotalDue) on in-store purchases in the USA. We used multiple JOIN operations to connect various tables and filter for physical store purchases (OnlineOrderFlag = 0) within the United States.
2. **Top30Cities**: We determined which cities these top 30 customers are located in. We used DISTINCT to avoid duplicates, as a customer might appear in multiple orders but we only needed their city once.
3. **TopOnlineCities**: We calculated total online sales revenue by city in the USA, explicitly excluding all cities where the top 30 in-store customers are located (using NOT IN). This gave us cities with high online sales potential but that would not directly compete with the best existing retail customers.
4. **Final Selection**: We selected the top 2 cities with the highest online revenue from among the cities that don't contain any of the top 30 in-store customers.

This approach ensures Adventure Works will avoid competing with their best retail customers while targeting areas with proven customer interest through online sales.

Analysis Implementation

The implementation used a series of temporary tables (prefixed with #) to progressively refine the data. We chose temporary tables for several reasons:

- **Improved Performance**: Temporary tables allow us to materialize intermediate results, reducing repeated complex joins and calculations
- **Clearer Code Structure**: Breaking the analysis into discrete steps makes the code more readable and maintainable
- **Easier Debugging**: Each temporary table can be inspected separately to verify results at each stage
- **Query Optimization**: SQL Server can better optimize execution plans for simpler queries against temporary tables

Step 1: Identify Top 30 Store Customers

In this step, we created a temporary table #Top30StoreCustomers to store the CustomerID and total spent amount for the top 30 customers who made physical store purchases in the United States. We joined several tables including SalesOrderHeader, BusinessEntityAddress, Address, StateProvince, and CountryRegion to filter for US-based orders with OnlineOrderFlag = 0, then grouped by CustomerID and ordered by the sum of TotalDue in descending order to select only the top 30.

Step 2: Identify Cities of Top Customers

We created the #Top30StoreCities temporary table to store the unique cities where these top 30 customers are located. We joined the previously created temporary table with multiple tables from the database to extract city information, using DISTINCT to ensure each city appeared only once in our results.

Step 3: Find Cities with Highest Online Sales

For this step, we created the #TopOnlineCities temporary table to identify and rank cities by their online sales revenue. We specifically targeted US cities where online sales were occurring (OnlineOrderFlag = 1) and explicitly excluded any cities that appeared in our #Top30StoreCities table using the NOT IN clause. We calculated total revenue by grouping the results by city.

Step 4: Select the 2 Best Cities

In the final step, we created the #RecommendedCities temporary table to store our final recommendations. We selected the top 2 cities from the #TopOnlineCities table,

ranking them by total online revenue in descending order. These cities represent the optimal locations for new brick and mortar stores based on our analysis criteria.

Analysis Results

The analysis identified two cities with the highest online sales revenue that do not contain any of the top 30 retail store customers. These cities represent the optimal locations for new Adventure Works brick and mortar stores based on demonstrated customer interest and minimal conflict with existing retail partners.

To validate the recommendations, additional analysis was performed:

1. Verification that none of the top 30 physical store customers are located in the recommended cities
2. Analysis of customer segments in the recommended cities (online vs. physical store)
3. Calculation of total customer count, revenue, and average order value in recommended cities

This data provides strong support for the final recommendations and offers additional context for the leadership team's decision-making process.

Conclusion

This project successfully delivered two key components to support Adventure Works' business goals:

1. A robust auction system to facilitate stock clearance before new product launches, with all required functionality and considerations for performance under high load conditions.
2. A data-driven recommendation for optimal locations to establish brick and mortar stores based on demonstrated customer interest while avoiding competition with key retail partners.

Both implementations follow best practices for SQL development, including error handling, performance optimization, and secure coding techniques. The auction system is ready for integration with the web front-end, and the store location recommendations provide actionable insights for business expansion.

Appendix: Schema Diagram

The diagram below illustrates the relational structure of the auction system's database tables and their relationships:

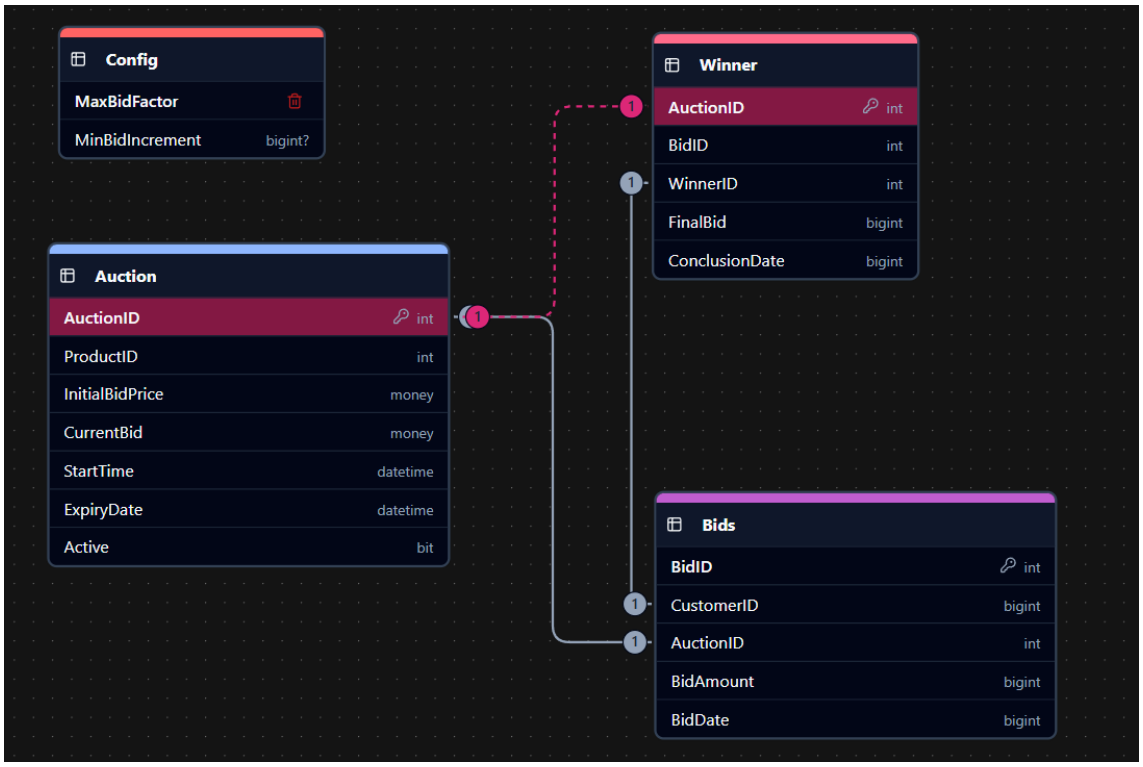

Figure 1

The schema demonstrates how the new Auction tables integrate with the existing Adventure Works database structure while maintaining clean separation through the dedicated schema.